

Chengxi Yang

✉ chengxi@berkeley.edu / 🏠 arcadia-y.github.io

🎓 EDUCATION

Shanghai Jiao Tong University

Bachelor of Computer Science

Shanghai, China

Sep. 2022 - Jun. 2026

- A member of ACM Honors Class, an elite CS program in SJTU
- **GPA: 92.7/100**, Ranking: 1/29, 22 A+ courses
- Selected Courses: Mathematical Logic: 100/100, Program Verification: 99/100, Model Checking: 100/100, Compiler Design and Implementation: 97/100, Cryptography: 100/100, Reinforcement Learning: 96/100

❓ RESEARCH INTEREST

My research interest mainly lies in **programming languages** and **formal methods**, aiming to develop elegant theories to solve real-world problems. I'm also broadly interested in other areas related to PL/FM, such as the intersection of AI and FM.

🔍 RESEARCH EXPERIENCE

University of Wisconsin-Madison

Research Intern at madPL, co-advised by Prof. Thomas Reps and Prof. Tej Chajed

Madison, WI, USA

Jun. 2025 - Dec. 2025

Fixing the Fixpoint: Sound Fixpoint Detection for Incremental Recursive Computation

As the project leader, I identified and formalized the **Fixpoint Detection** (FPD) problem in incremental recursive computation frameworks such as DBSP, exposing a semantic gap between theory and implementation. I proved that the commonly suggested **FirstZero** criterion is unsound for delta-of-deltas incrementalization and that exact FPD is undecidable in general. I then formalized **internal convergence** (IntConv), proving its soundness, and completeness for regular circuits and their incrementally optimized forms, with key results mechanized in Lean. Submitted to **OOPSLA 2026**; currently under review.

Shanghai Jiao Tong University

Research Assistant, advised by Prof. Qinxiang Cao

Shanghai, China

Jun. 2024 - Jun. 2026

Formalizing Temporal Reasoning Using the Trace Method

I proposed a “**trace method**” to formalize temporal reasoning about program history, which is hard to capture in standard Hoare logic. I introduced novel proof rules for recursions and loops that faithfully reflect natural reasoning patterns. We have used this to formalize algorithms like DFS and BFS algorithm in Rocq.

A Formal Framework for Naturally Specifying and Verifying Algorithms

I built a formal framework in Rocq, including a Hoare logic and tactics, for more natural algorithm verification. I proposed a novel **two-stage proof approach** that separates high-level logical reasoning from mechanical proof composition, better mirroring the logical structure of natural proofs. The paper is published in TASE 2025.

Contributing to the C Verification Tool QCP

I designed some core features for the QCP C verification tool, including “**multi-branch**”, which enable fine-grained control over branches in symbolic execution. I also designed, implemented, and now maintain the tool's **VS Code extension** to improve usability, and I have contributed verifications of several examples to its backend Rocq library.

📖 PUBLICATIONS AND PREPRINTS

Fixing the Fixpoint: A Sound Convergence Criterion for Incremental Recursive Computation

Chengxi Yang, Tej Chajed, Thomas Reps

Submitted to OOPSLA 2026; currently under review.

A Formal Framework for Naturally Specifying and Verifying Sequential Algorithms

Chengxi Yang, Shushu Wu*, Qinxiang Cao*

In Theoretical Aspects of Software Engineering (TASE) 2025.

Assertions for Free: Transferring Invariants from Algorithm to Implementation Proofs (Functional Pearl)

Shushu Wu, Chengxi Yang, Xiwei Wu, Qinxiang Cao

Accepted at ICFP 2026.

QCP: A Practical Separation Logic-based C Program Verification Tool

Xiwei Wu, Yueyang Feng, Xiaoyang Lu*, Tianchuan Lin, Kan Liu, Zhiyi Wang, Shushu Wu, Lihan Xie, Chengxi Yang, Hongyi Zhong, Naijun Zhan, Zhenjiang Hu, Qinxiang Cao*

In TASE 2026.

Intuitive Verification of Sequential Programs Using Hybrid Reasoning

Shushu Wu, Xiwei Wu, Chengxi Yang, Qinxiang Cao

In TASE 2026.

SELECTED PROJECTS

- **Mx* Compiler:** A Compiler from Mx* language (a C & Java like language) to RISC-V RV32I Assembly implemented in Java, featuring linear scan register allocation, SSA transformation and various optimization.
- **LTL Model Checker:** A Linear Temporal Logic (LTL) model checker implemented in Rust. Given a transition system (TS) and an LTL formula, it can check whether the TS or its arbitrary state satisfies the formula.
- **RISC-V CPU:** A Tomasulo-based RISC-V RV32I CPU designed in Verilog RTL, featuring instruction cache and branch predictor. It can be run on FPGA board.
- **Acore:** A toy RISC-V kernel implemented in Rust that can be run on QEMU. It features remote-procedure-call mechanism, virtual memory and preemptive multi-process scheduling.
- **Resource-Constrained MOBA Agent:** We optimized PPO-based agents for Honor of Kings under limited computational resources by making reward dynamic and denser, integrating Phasic Policy Gradient (PPG) and mixed opponent selection.

HONORS & AWARDS

Scholarship

- **OPPO “Benfen” Scholarship** (first-year winner, about 30 winners in SJTU), 2025
- **Fan Xuji Scholarship** (about 15 winners each year in SJTU), 2024
- **Zhiyuan Honorary Scholarship** (awarded to top 10% students in SJTU annually), 2022-2025

TEACHING

- **TA for Programming(C++).** I gave a lecture on introduction to computer systems, designed a new course project and a problem in the final exam.
- **TA for Principles and Practice of Computer Algorithms.** I designed a new lab project on formal verification in Coq.
- **TA for Mathematical Logic.** I answered students’ questions, graded their homework and wrote sample solutions for them.
- **TA for Program Verification.** I maintained the course git repository and participated in the design of the final project.

SKILLS

Natural Languages: Chinese (native), English (fluent, TOEFL 108).

Programming Languages: Rocq (Coq), Lean, Rust, C/C++, Java, Python, Verilog, Go.